Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

第十章 博客网站简介



成都信息工程大學
Chengdu University of Information Technology

- 功能结构
- 网站首页
- 用户管理
- 博客管理
- 评论管理

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署

REST

REST概述

- 近些年，Web 应用有种趋势，那就是业务逻辑被越来越多地移到客户端，开创出了一种称为富互联网应用（RIA，rich Internet application）的架构，服务器的主要功能（有时是唯一功能）是为客户端提供数据存取服务，服务器变成了 Web 服务或应用编程接口
- RIA 可采用多种协议与 Web 服务通信：远程过程调用(RPC)协议、简单对象访问协议(SOAP)、表现层状态转移(REST)架构
- REST特性：客户端–服务器、无状态、缓存、接口统一、系统分层、按需编程
- 资源：是REST架构风格的核心概念（用户、博客文章和评论），每个资源、资源集合、资源的逻辑子集都要使用唯一的URL表示
- 请求方式：GET(获取)、POST(创建)、PUT(修改)、DELETE(删除)
- 请求和响应主体：请求和响应中的Content-Type首部指明主体中资源的编码方式。常用的两种编码方式：JavaScript 对象表示法(JSON)和可扩展标记语言(XML)

REST

REST概述

- 对基于 Web 的 RIA 来说，JSON 更具吸引力，因为 JSON 比 XML 简洁，而且 JSON 与 Web 浏览器使用的客户端脚本语言 JavaScript 联系紧密

```
{
    "self_url": "http://www.example.com/api/posts/12345",
    "title": "Writing RESTful APIs in Python",
    "author_url": "http://www.example.com/api/users/2",
    "body": "... text of the article here ...",
    "comments_url": "http://www.example.com/api/posts/12345/comments"
}
```

https://www.runoob.com/json/json-tutorial.html

REST

REST概述

JSON

- ✓ 是 JavaScript 对象表示法（JavaScript Object Notation）
- ✓ 是轻量级的文本数据交换格式
- ✓ 独立于语言：JSON 使用 Javascript语法来描述数据对象，但是 JSON 仍然独立于语言和平台。JSON 解析器和 JSON 库支持许多不同的编程语言
- ✓ 具有自我描述性，更易理解

JSON语法规则

- ✓ 数据在名称/值对中；数据由逗号 , 分隔；使用斜杆 \ 来转义字符
- ✓ 大括号 {} 保存对象；中括号 [] 保存数组，数组可以包含多个对象

JSON常用函数

- ✓ JSON.parse()　　数据转换为 JavaScript 对象
- ✓ JSON.stringify()　　将 JavaScript 对象转换为字符串

https://www.runoob.com/json/json-tutorial.html

REST

REST实现

- 使用 Flask 创建 REST 式 Web 服务十分简单。使用熟悉的route()装饰器及其methods可选参数可以声明服务所提供资源 URL 的路由
- 处理JSON数据同样简单，请求中的JSON数据可以通过request.get_json()转换成字典格式，而且可以使用 Flask 提供的辅助函数jsonify()，从 Python 字典中生成需要包含 JSON 的响应

```
@main.app_errorhandler(404)
def page_not_found(e):
    if request.accept_mimetypes.accept_json and \
            not request.accept_mimetypes.accept_html:
        response = jsonify({'error': 'not found'})
        response.status_code = 404
        return response
    return render_template('404.html'), 404
```

```
def forbidden(message):
    response = jsonify({'error': 'forbidden', 'message': message})
    response.status_code = 403
    return response
```

REST实现

- 序列化，内部表示转换成传输格式的过程，字典->JOSN

```
class Post(db.Model):
    # ...
    def to_json(self):
        json_post = {
            'url': url_for('api.get_post', id=self.id),
            'body': self.body,
            'body_html': self.body_html,
            'timestamp': self.timestamp,
            'author_url': url_for('api.get_user', id=self.author_id),
            'comments_url': url_for('api.get_post_comments', id=self.id),
            'comment_count': self.comments.count()
        }
        return json_post
```

```
class User(UserMixin, db.Model):
    # ...
    def to_json(self):
        json_user = {
            'url': url_for('api.get_user', id=self.id),
            'username': self.username,
            'member_since': self.member_since,
            'last_seen': self.last_seen,
            'posts_url': url_for('api.get_user_posts', id=self.id),
            'followed_posts_url': url_for('api.get_user_followed_posts',
                                          id=self.id),
            'post_count': self.posts.count()
        }
        return json_user
```

- 反序列化，传输格式转换成内部表达的过程，JOSN->字典

```
from app.exceptions import ValidationError

class Post(db.Model):
    # ...
    @staticmethod
    def from_json(json_post):
        body = json_post.get('body')
        if body is None or body == '':
            raise ValidationError('post does not have a body')
        return Post(body=body)
```

```
@api.errorhandler(ValidationError)
def validation_error(e):
    return bad_request(e.args[0])
```

```
@api.route('/posts/', methods=['POST'])
def new_post():
    post = Post.from_json(request.json)
    post.author = g.current_user
    db.session.add(post)
    db.session.commit()
    return jsonify(post.to_json())
```

REST

REST实现

资源端点的实现

GET

```
@api.route('/posts/')
def get_posts():
    posts = Post.query.all()
    return jsonify({ 'posts': [post.to_json() for post in posts] })
```

```
@api.route('/posts/<int:id>')
def get_post(id):
    post = Post.query.get_or_404(id)
    return jsonify(post.to_json())
```

POST

```
@api.route('/posts/', methods=['POST'])
@permission_required(Permission.WRITE)
def new_post():
    post = Post.from_json(request.json)
    post.author = g.current_user
    db.session.add(post)
    db.session.commit()
    return jsonify(post.to_json()), 201, \
        {'Location': url_for('api.get_post', id=post.id)}
```

PUT

```
@api.route('/posts/<int:id>', methods=['PUT'])
@permission_required(Permission.WRITE)
def edit_post(id):
    post = Post.query.get_or_404(id)
    if g.current_user != post.author and \
            not g.current_user.can(Permission.ADMIN):
        return forbidden('Insufficient permissions')
    post.body = request.json.get('body', post.body)
    db.session.add(post)
    db.session.commit()
    return jsonify(post.to_json())
```

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署

其他扩展

- 为 Flask 应用增加功能，帮助完成公共任务的包，比如增加发送电子邮件或者连接数据库中的功能，Flask-SQLAlchemy为在Flask中轻松使用SQLAlchemy提供支持。有些扩展还有助于为应用添加全新的框架，如REST API
- 寻找扩展：https://pypi.org/
- 使用扩展
  - 下载安装： pip install flask-扩展名
  - 实例化和初始化：foo = Foo(app)　　foo = Foo()/foo.init_app(app)
  - 扩展使用：foo.method()/foo.attribute

其他扩展

- Flask-Mail　　用于实现邮件发送功能的扩展
  - 安装：　　pip install Flask-Mail
  - 主要参数设置：
  - 核心类：
    - ✓ Mail　管理电子邮件消息传递需求
      - send() 发送Message类对象的内容
      - connect() 打开与邮件主机的连接
      - send_message() 发送消息对象
    - ✓ Message　封装了一封电子邮件
      - attach() 为邮件添加附件
      - add_recipient() 添加收件人

参数	描述
MAIL_SERVER	电子邮件服务器的名称/IP地址
MAIL_PORT	使用的服务器的端口号
MAIL_USE_TLS	启用/禁用传输安全层加密
MAIL_USE_SSL	启用/禁用安全套接字层加密
MAIL_DEBUG	调试支持。默认值是Flask应用程序的调试状态
MAIL_USERNAME	发件人的用户名
MAIL_PASSWORD	发件人的密码
MAIL_DEFAULT_SENDER	设置默认发件人
MAIL_MAX_EMAILS	设置要发送的最大邮件数
MAIL_SUPPRESS_SEND	如果app.testing设置为true，则发送被抑制
MAIL_ASCII_ATTACHMENTS	如果设置为true，则附加的文件名将转换为ASCII

Mail(app)
Message(subject, recipients, body, html, sender, date,**kwargs)

https://blog.csdn.net/shifengboy/article/details/114274271

其他扩展

- Flask-Mail　　用于实现邮件发送功能的扩展

```python
from flask import Flask
from flask_mail import Mail, Message

app =Flask(__name__)
mail=Mail(app)

app.config['MAIL_SERVER']='smtp.qq.com'
app.config['MAIL_PORT'] = 465
app.config['MAIL_USERNAME'] = 'xxxxx@qq.com'
app.config['MAIL_PASSWORD'] = 'xxxxxxxx'
app.config['MAIL_USE_TLS'] = False
app.config['MAIL_USE_SSL'] = True
mail = Mail(app)

@app.route("/")
def index():
   msg = Message('Hello', sender = 'xxxxx@qq.com', recipients = ['xxxxx@qq.com'])
   msg.body = "Hello Flask message sent from Flask-Mail"
   mail.send(msg)
   return "Sent"

if __name__ == '__main__':
   app.run(debug = True)
```

https://blog.csdn.net/shifengboy/article/details/114274271

其他扩展

- Flask-Login　　处理登录、注销和长时间记住用户会话等常见任务
  - 安装　　pip install flask-login
  - 核心类:
    - ✓ LoginManager　　管理用户登录相关内容
      - login-view：验证失败跳转的界面
      - login-message：用户重定向到登录页面时闪出的消息
      - refresh-view：用户需要重新进行身份验证时要重定向到的视图的名称
      - needs-refresh-message：用户重定向到 “需要刷新” 页面时闪出的消息
      - session-protection：使用会话保护的模式
      - user_loader：自定义回调函数
      - login_required：确保在调用实际视图之前登录并验证当前用户
      - login_user：记录/保存当前成功登陆的用户
      - logout_user：登出功能

```
@login_manager.user_loader
def load_user(userid):
    return User.get(userid)
```

https://blog.csdn.net/be_racle/article/details/128081618

其他扩展

- Flask-Login　　处理登录、注销和长时间记住用户会话等常见任务
  - 核心类：
    - ✓ UserMixin　　一般作为用户类的父类
      - is_authenticated：当用户通过验证时，也即提供有效证明时返回
      - is_active：如果这是一个活动用户且通过验证，账户也已激活，未被停用，返回 True
      - is_anonymous：如果是一个匿名用户，返回 True
      - get_id()：返回一个能唯一识别用户的，并能用于从 user_loader 回调中加载用户的 unicode

其他扩展

- Flask-Moment 管理时间日期等信息
  - 安装与配置： pip install flask-moment ， 前端配置支持库moment.js，一般在父模板中添加 {{ moment.include_moment() }}
  - 核心类：
    - moment
      - include_moment() 文档中引入moment.js
      - locale() 设置本地语言，默认为英文， ‘zh-ch’ 为中文
      - format() 设置时间显示格式
      - fromNow() /fromTime() 返回一个相对时间，与传到前端的时间点对应
      - toNow()/toTime() 返回一个相对时间，计算到现在的时间
      - calendar() 返回一个相对的日期
      - valueOf()/unix() 返回毫秒/秒

https://blog.csdn.net/weixin_45950544/article/details/104109913

其他扩展

- flask-principal 管理角色、权限与授权库
- Flask-Security 管理用户认证和授权扩展库
- Flask-Admin 后台管理库
- Flask-Session 处理用户会话（Session）管理
- Flask-Bable 提供了基于Babel库的多语言支持
- Flask-marshmallow 序列化和反序列化模块
- Flask-PyMongo 操作MongoDB
- Flask-DebugToolbar 帮助调试和优化你的Flask应用程序
- Flask-Uploads 优化了文件上传

https://blog.csdn.net/zhouruifu2015/article/details/133387505

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署

测试

软件测试

https://zhuanlan.zhihu.com/p/627063652?utm_id=0

- 使用技术手段验证软件是否满足使用需求，其目的在于减少软件缺陷（BUG），保障软件质量
- 主流技术：
  - ✓ 功能测试：测试主要验证程序的功能是否满足需求
  - ✓ 自动化测试：使用代码或工具代替手工进行测试
  - ✓ 接口测试：使用代码或工具对服务提供的接口进行测试
  - ✓ 性能测试：模拟多人使用软件，查找软件服务器缺陷，一般使用工具实现
- 分类
  - ✓ 按测试阶段：单元测试、集成测试、系统测试、验收测试
  - ✓ 按代码可见度：黑盒测试-功能测试、灰盒测试-接口测试、白盒测试-单元测试
- 质量模型：功能性、性能性、兼容性、安全性、可靠性、可移植性、可维护性

前置条件	测试步骤	测试数据	预期结果
1、打开登录页面 2、手机已注册	1、输入手机号 2、点击发送验证码按钮 3、拖拽滑块成功 4、输入验证码 5、勾选协议 6、点击登录	1、手机：已注册手机号 2、其他输入正确数据	1、手机文本框状态为绿色； 2、登录成功，跳转到主页；
1、打开登录页面	1、输入手机号 2、点击发送验证码按钮 3、输入验证码 4、勾选协议 5、点击登录	1、手机：为空 2、验证码：8888 3、其他输入正确数据	1、手机文本框状态为红色； 2、点击发送验证码按钮，不弹出滑块； 3、登录失败，提示手机号不能为空；
1、打开登录页面	1、输入手机号 2、点击发送验证码按钮 3、输入验证码 4、勾选协议 5、点击登录	1、手机：1380000111 2、验证码：8888 3、其他输入正确数据	1、未收到验证码短信； 2、手机号文本框为红色 3、登录失败，提示：手机号格式不正确；

测试

软件测试

https://zhuanlan.zhihu.com/p/627063652?utm_id=0

测试流程

- ✓ 需求评审：确保对需求理解一致
- ✓ 计划编写：为测试做详细的计划
- ✓ 用例设计：验证项目是否符合需求的操作文档
- ✓ 用例执行：对系统进行用例的实施
- ✓ 缺陷管理：对测试过程中发现的问题进行规范管理
- ✓ 测试报告：对整个测试过程的总结

测试

Web应用的测试内容

- 功能测试
  - ✓ 链接测试
  - ✓ 表单测试
  - ✓ 数据校验
  - ✓ Cookies测试
  - ✓ 数据库测试
  - ✓ 特定功能测试
  - ✓ 设计语言测试
- 性能测试
  - ✓ 连接速度测试
  - ✓ 负载测试
  - ✓ 压力测试
- 用户界面测试
  - ✓ 导航测试
  - ✓ 图形测试
  - ✓ 内容测试
  - ✓ 表格测试
  - ✓ 整体界面测试
- 兼容性测试
  - ✓ 浏览器测试
  - ✓ 分辨率测试
  - ✓ FTTH/FTTP连接速度
  - ✓ 打印机
  - ✓ 组合测试
- 安全测试
  - ✓ SSL
  - ✓ 登录
  - ✓ 日志文件
  - ✓ 脚本语言
- 接口测试
  - ✓ 服务器端接口
  - ✓ 外部接口
  - ✓ 错误处理

测试

Unittest

- Unittest是Python内置的一个测试框架，它支持测试自动化和单元测试
- 它提供了一组用于编写和运行测试的工具和库，包括测试装置、测试套件和测试运行器
- 基本组成：
  - ✓ 测试用例(TestCase)：测试的最小单位，包含一个或多个测试方法，需要继承 unittest.TestCase 类，测试方法的名称必须以 test_ 开头
  - ✓ 测试装置(Test Fixture)：用于设置和清理测试环境，创建临时或代理的数据库、目录，再或者启动一个服务器进程，包括 setUp() 、 tearDown() 、 setUpClass() 、 tearDownClass()方法
  - ✓ 测试套件(Test Suite)：测试用例的集合
  - ✓ 测试运行器(Test Runner)：用于执行测试用例和生成测试报告，包括 TextTestRunner、HTMLTestRunner 等

https://docs.python.org/3/library/unittest.html

https://zhuanlan.zhihu.com/p/612683146

测试

Unittest

基本实例

```python
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

命令行接口

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

https://docs.python.org/3/library/unittest.html

https://zhuanlan.zhihu.com/p/612683146

测试

- Unittest
  - 常用的断言

方法	检查对象
assertEqual(a, b)	a == b
assertNotEqual(a, b)	a != b
assertTrue(x)	bool(x) is True
assertFalse(x)	bool(x) is False
assertIs(a, b)	a is b
assertIsNot(a, b)	a is not b
assertIsNone(x)	x is None
assertIsNotNone(x)	x is not None
assertIn(a, b)	a in b
assertNotIn(a, b)	a not in b
assertIsInstance(a, b)	isinstance(a, b)
assertNotIsInstance(a, b)	not isinstance(a, b)

方法	检查对象
assertRaises(exc, fun, *args, **kwds)	fun(*args, **kwds) 引发了 *exc*
assertRaisesRegex(exc, r, fun, *args, **kwds)	fun(*args, **kwds) 引发了 *exc* 并且消息可与正则表达式 *r* 相匹配
assertWarns(warn, fun, *args, **kwds)	fun(*args, **kwds) 引发了 *warn*
assertWarnsRegex(warn, r, fun, *args, **kwds)	fun(*args, **kwds) 引发了 *warn* 并且消息可与正则表达式 *r* 相匹配
assertLogs(logger, level)	with 代码块在 *logger* 上使用了最小的 *level* 级别写入日志
assertNoLogs(logger, level)	with 代码块没有在 *logger* 上使用最小的 *level* 级别写入日志

方法	用作比较
assertMultiLineEqual(a, b)	字符串
assertSequenceEqual(a, b)	序列
assertListEqual(a, b)	列表
assertTupleEqual(a, b)	元组
assertSetEqual(a, b)	集合
assertDictEqual(a, b)	字典

方法	检查对象
assertAlmostEqual(a, b)	round(a-b, 7) == 0
assertNotAlmostEqual(a, b)	round(a-b, 7) != 0
assertGreater(a, b)	a > b
assertGreaterEqual(a, b)	a >= b
assertLess(a, b)	a < b
assertLessEqual(a, b)	a <= b
assertRegex(s, r)	r.search(s)
assertNotRegex(s, r)	not r.search(s)
assertCountEqual(a, b)	*a* 和 *b* 具有同样数量的相同元素，无论其顺序如何。

https://docs.python.org/3/library/unittest.html

https://zhuanlan.zhihu.com/p/612683146

测试

Unittest

合理的组织测试代码

- ✓ 单元测试的构建单位是测试用例：独立的、包含执行条件与正确性检查的方案
- ✓ 可能同时存在多个前置操作相同的测试，我们可以把测试的前置操作从测试代码中拆解出来，并实现测试前置方法，在运行测试时，测试框架会自动地为每个单独测试调用前置方法
- ✓ 在测试运行时，若 setUp() 方法引发异常，测试框架会认为测试发生了错误，因此测试方法不会被运行。提供了一个 tearDown() 方法在测试方法运行后进行清理工作
- ✓ 根据用例所测试的功能将测试用 TestCase 分组

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

```
def suite():
    suite = unittest.TestSuite()
    suite.addTest(WidgetTestCase('test_default_widget_size'))
    suite.addTest(WidgetTestCase('test_widget_resize'))
    return suite

if __name__ == '__main__':
    runner = unittest.TextTestRunner()
    runner.run(suite())
```

- ✓ 跳过单个或整组的测试用例
  - ✓ @unittest.skip(reason)
  - ✓ @unittest.skipIf(condition, reason)
  - ✓ @unittest.skipUnless(condition, reason)

```
@unittest.skip("demonstrating skipping")
def test_nothing(self):
    self.fail("shouldn't happen")
```

https://docs.python.org/3/library/unittest.html

https://zhuanlan.zhihu.com/p/612683146

测试

Pytest

- 是一非常成熟的python单元框架，比unittest更灵活，容易上手
- 可以和selenium、requests、appium结合实现web自动化，接口自动化，app自动化
- 可以实现测试用例的跳过以及reruns失败用例重试
- 可以和allure生成非常美观的测试报告
- 可以和jenkins持续集成
- 有很多非常强大的插件，并且这些插件能够实现很多的实用的操作
  - ✓ pytest-xdist 测试用例分布式执行，多cpu并发
  - ✓ pytest-ordering 用于改变测试用例的执行顺序
  - ✓ pytest-rerunfailures 用于失败后重跑
  - ✓ pytest-html （生成html格式的自动化测试报告）
  - ✓ allure-pytest 用于生成美观的测试报告

https://www.osgeo.cn/pytest/contents.html

https://blog.csdn.net/m0_65657661/article/details/129238306

测试

Pytest

使用pytest，默认的测试用例的规则

- ✓ 模块名必须以test_开头或以_test结尾 （.py文件）
- ✓ 测试类必须以Test开头，并且不能有__init__方法（class 定义的类名）
- ✓ 测试方法必须以test开头 （def 定义的函数名）
- ✓ 默认从当前目录下收集测试用例，当配置testpath时，从配置的路径下收集测试用例
- ✓ 注：修改配置文件后，模块名、测试类、测试方法应该与配置文件一致

基本使用

- ✓ 创建测试用例： 函数

```
def test_01():
    assert 1 == 2
```

类

```
class TestPytest():
    def test_01(self):
        assert 1 == 1

    def test_02(self):
        assert 1 == 2
```

- ✓ 执行测试用例：
  - 命令行中使用 pytest、 pytest -vs 模块名[[::类名]::方法名]
  - 模块中执行pytest.main()

https://www.osgeo.cn/pytest/contents.html

https://blog.csdn.net/m0_65657661/article/details/129238306

测试

- Pytest
  - 用例执行顺序
    - ✓ 在模块级别采用模块名的ascii码顺序
    - ✓ 在模块内部根据从上往下的定义顺序来执行
    - ✓ 可使用mark标记改变pytest默认的执行顺序

```python
class TestInterface:
    def test_03_interface(self):
        print('test03')

    @pytest.mark.run(order=2) #定义运行顺序
    def test_05_interface(self):
        print('test-05')

    @pytest.mark.run(order=1) #定义运行顺序
    def test_06_interface(self):
        print('test-06')

    def test_07_interface(self):
        print('test-07')
#执行顺序为6-5-3-7
```

  - 在pytest中只需使用python语言标准的断言语句 assert 来断言
  - 前后置断言
    - ✓ 经典的xunit风格
      - 模块级别的setup_module和teardown_module
      - 类级别setup_class和teardown_class(使用@classmethod)
      - 方法级别setup_method和teardown_method(使用@classmethod)
      - 方法级别setup_function和teardown_function
    - ✓ unittest风格
    - ✓ 装饰器
      - 先定义 @pytest.fixture
      - 调用@pytest.mark.usefixtures()

https://www.osgeo.cn/pytest/contents.html

https://blog.csdn.net/m0_65657661/article/details/129238306

测试

- Pytest
  - pytest.ini配置文件
    - ✓ 是pytest单元测试框架的核心配置文件，需删除注释
    - ✓ 一般放在项目的根目录
    - ✓ 编码格式必须是ANSI
    - ✓ 用于改变pytest默认的行为/规则

```
[pytest]
addopts=-vs                    #命令行的参数，可以输入多个，用空格分隔，主函数和命令行模式处直接执行
testpaths=./testcase           #测试用例的路径
python_files=test_*.py         #配置测试搜索模块名的规则
python_classes=Test            #配置测试搜索类名的规则
python_functions=test          #配置测试搜索方法名的规则（函数）
markers=                       #标记的模块
    smoke:冒烟用例
    uermanage:用户管理模块
    productmanage:商品管理模块
```

  - 分组执行
    - ✓ 在配置文件中标记模块
    - ✓ 在用例函数处使用@pytest.mark.标记名装饰器
    - ✓ 在主函数中使用 –m 参数配置执行分组
  - pytets跳过用例
    - ✓ 无条件跳过：@pytest.mark.skip
    - ✓ 有条件跳过：@pytest.mark.skipif(条件)

https://www.osgeo.cn/pytest/contents.html

https://blog.csdn.net/m0_65657661/article/details/129238306

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署

部署

部署流程

- 架设服务器
  - ✓ 安装数据库服务器，例如 MySQL
  - ✓ 安装适用于生产环境的 Web 服务器，例如 Gunicorn 或 uWSGI
  - ✓ 安装一个进程监控工具，例如 Supervisor，在服务器崩溃或恢复电力后立即重启
  - ✓ 为了启用安全的 HTTP，安装并配置 SSL 证书
  - ✓ 安装前端反向代理服务器，例如 nginx 或 Apache
  - ✓ 提升服务器的安全性。目标在于降低服务器被攻击的概率，例如安装防火墙、删除不用的软件和服务，等等
- 导入环境变量，运行在独立服务器上的应用也要通过环境变量做些设置，例如数据库连接 URL、电子邮件服务器凭据，
- 配置日志

部署

- 部署流程-远程用开发版/自带的服务器
  - 购置服务器或云服务器
  - 在云服务器上完成环境部署(安装操作系统、python、虚拟环境、数据库等)
  - 利用pyCharm将本地代码与云服务器进行同步
  - 虚拟环境同步-requirements.txt、数据库同步-Navicat-数据传输、修改项目配置
  - 开启云服务器的5000端口
  - python app.py runserver –h 0.0.0.0 –p 5000
  - 访问

https://www.bilibili.com/video/BV17z4y1X7UZ?p=48/49

https://blog.csdn.net/qq_45100200/article/details/130355935

部署

- 部署流程-uWSGI
  - 购置服务器或云服务器
  - 在云服务器上完成环境部署(安装操作系统、python、虚拟环境、数据库等)
  - 搭建稳定服务器(安装、配置Nginx、uWSGI、Redis)
  - 启动 (nginx –c nginx.conf)
  - 启动 (uwsgi –ini uwsgi.ini)
  - 访问
  - 停止uWSGI(uwsgi –stop uwsgi.pid)
  - 停止Nginx(nginx -s stop)

https://www.bilibili.com/video/BV17z4y1X7UZ?p=50-54

https://zhuanlan.zhihu.com/p/83890573

https://zhuanlan.zhihu.com/p/366440889

部署

- 部署流程-云部署-Heroku平台
  - 将项目存入Git仓库
  - 准备工作：注册Heroku账户、安装Heroku CLI、创建应用、配置数据库、配置日志、配置电子邮件、添加顶层需求文件、使用Flask-SSLify启用安全的HTTP、运行Web生产服务器（Gunicorn 或 uWSGI）、添加Procfile文件
  - 使用heroku local 测试，heroku local:run flask deploy
  - 执行git push 命令部署，git push heroku master、heroku run flask deploy 、heroku restart
  - https://<appname>.herokuapp.com 访问

部署

- 部署流程-云部署-Docker容器
  - 安装Docker
  - 构建容器映像，docker build -t flasky:latest
  - 运行容器 docker run -d -p 8000:5000 --link mysq1 :dbserver

    -e DATABASE_URL=mysql+pymysq1: //f1asky :<database-password>@dbserver/flasky

    -e MAIL_USERNAME=<your-gmai1-username>

    -e MAIL_PASSWORD=<your-gmai1-password>flasky:latest

第十一章 REST、其他扩展、测试及部署



成都信息工程大學
Chengdu University of Information Technology

- REST
- 其他扩展
- 测试
- 部署